ESPECIFICACIONES TÉCNICAS

Anexo 2 — Contrato N° FINU-2026-001

FinanzasU — Plataforma Web de Gestión Financiera Personal

Campo	Detalle
Documento	Especificaciones Técnicas — Anexo 2
Proyecto	FinanzasU — Plataforma Web de Gestión Financiera
Contrato	FINU-2026-001
Cliente	FinanzasU S.A.S. — Juan Camilo Triana
Desarrollador	Rise — Deibyd Castillo
Versión	1.0
Fecha	___ / ___ / 2026
Supervisor académico	Gustavo Adolfo Osorio — Universidad del Valle, Sede Zarzal

1. Introducción

El presente documento define las especificaciones técnicas de la plataforma web FinanzasU, desarrollada por Rise para FinanzasU S.A.S. en el marco del Contrato N° FINU-2026-001. Establece los requisitos funcionales, no funcionales, la arquitectura del sistema, el modelo de datos y los criterios técnicos que el equipo desarrollador Rise debe cumplir para dar por terminado el proyecto.

> **Propósito del documento:**
> Servir como referencia técnica vinculante entre EL CLIENTE (FinanzasU S.A.S.) y EL DESARROLLADOR (Rise) durante los 5 sprints de desarrollo. Cualquier cambio al alcance técnico aquí definido debe acordarse por escrito entre ambas partes antes de ser implementado.

2. Descripción General del Sistema

2.1. Visión del producto

FinanzasU es una plataforma web de gestión financiera personal diseñada exclusivamente para estudiantes universitarios. Permite registrar ingresos y gastos, establecer presupuestos mensuales por categoría con alertas automáticas y visualizar estadísticas interactivas de hábitos de consumo, todo desde cualquier navegador moderno sin necesidad de instalación ni vinculación bancaria.

2.2. Usuarios del sistema

Tipo de usuario	Descripción	Permisos

Estudiante autenticado	Estudiante universitario registrado en la plataforma con correo y contraseña	Acceso total a sus propios datos: transacciones, categorías, presupuestos, dashboard y perfil
Usuario no autenticado	Visitante que accede a la URL sin sesión activa	Solo puede ver las páginas de login y registro. Redirigido automáticamente si intenta acceder a rutas protegidas

2.3. Restricciones del sistema

- La plataforma no requiere ni solicita datos bancarios en ningún momento.
- No existe rol de administrador en el MVP — cada usuario solo gestiona sus propios datos.
- El sistema no incluye notificaciones push ni por correo electrónico en esta versión.
- No se integra con servicios de pago ni con APIs bancarias externas.
- Las categorías del sistema son de solo lectura para todos los usuarios.

3. Stack Tecnológico

Capa	Tecnología	Versión	Rol en el proyecto
Frontend	React	18+	Framework de interfaz de usuario con componentes funcionales y hooks
Bundler	Vite	5+	Servidor de desarrollo con HMR y build optimizado para producción
Estilos	Tailwind CSS	3+	Framework CSS utility-first para diseño responsive sin CSS personalizado
Gráficas	Recharts	2+	Librería de gráficas para React: torta, barras y línea
Backend & DB	Supabase	Latest	PostgreSQL + Auth + API REST autogenerada + Edge Functions + Storage
Base de datos	PostgreSQL	15+	Motor relacional gestionado por Supabase con soporte completo de SQL
Autenticación	Supabase Auth	Latest	Gestión de sesiones con tokens JWT y hash bcrypt en contraseñas
Seguridad DB	RLS Supabase	—	Row Level Security — políticas de acceso a datos por usuario
Cliente Supabase	@supabase/supabase-js	2+	SDK oficial de Supabase para React
Control versiones	Git + GitHub	—	Versionado de código con flujo main → develop → feature/HU-XX

3.1. Herramientas de desarrollo

Herramienta	Uso
VS Code	Editor de código principal del equipo
Postman	Pruebas de endpoints de Supabase y funciones RPC
Supabase Studio	Gestión visual de la base de datos, políticas RLS y logs
GitHub Projects	Tablero Kanban para gestión de historias de usuario por sprint
Figma	Diseño de mockups de interfaz antes de implementar

4. Arquitectura del Sistema

4.1. Diagrama de arquitectura

Flujo de datos:
Navegador → React (componentes + hooks) → supabaseClient.js → Supabase API REST / RPC → PostgreSQL (con RLS). La autenticación se gestiona con tokens JWT que Supabase valida automáticamente en cada petición. No existe un servidor backend propio: toda la lógica de datos se delega a Supabase.

4.2. Estructura de carpetas del frontend

```
src/ ├── components/      # Componentes reutilizables (UI, layout, gráficas) │   ├── layout/      # Navbar.jsx, Layout.jsx, ProtectedRoute.jsx │   ├── ui/              # Modal.jsx, ProgressBar.jsx, Spinner.jsx, AlertBadge.jsx │   └── charts/       # PieChart.jsx, BarChart.jsx, LineChart.jsx ├── pages/         # Una página por módulo: Login, Register, Dashboard, │                 # Transacciones, Categorias, Presupuestos, Perfil ├── hooks/         # Custom hooks: useAuth, useTransacciones, useCategorias, │                 # usePresupuestos, useDashboard ├── services/        # supabaseClient.js + un service por entidad ├── utils/           # formatMoneda.js, exportCSV.js, constants.js ├── App.jsx          # Router con rutas protegidas vía ProtectedRoute └── main.jsx # Punto de entrada
```

4.3. Patrón de arquitectura

Capa	Responsabilidad	Ejemplo
Pages	Orquestar la UI y los hooks — sin lógica de datos directa	Dashboard.jsx llama a useDashboard()
Hooks	Manejar estado, efectos y llamadas a services	useTransacciones.js maneja el CRUD y el estado local
Services	Ejecutar queries a Supabase — sin estado propio	transaccionesService.js llama a supabase.from('transacciones')
Supabase	Ejecutar SQL, aplicar RLS y retornar JSON	PostgreSQL valida que usuario_id = auth.uid()

5. Modelo de Datos

5.1. Tablas del sistema

Tabla: categorias

Columna	Tipo	Restricciones	Descripción
id	UUID	PK, DEFAULT gen_random_uuid()	Identificador único de la categoría
nombre	VARCHAR(80)	NOT NULL	Nombre de la categoría (ej: Comida, Transporte)
tipo	VARCHAR(10)	NOT NULL, CHECK IN ('ingreso','gasto')	Clasifica si aplica a ingresos o gastos
usuario_id	UUID	FK auth.users, NULL = predeterminada	NULL para categorías del sistema; UUID del usuario para las personalizadas
icono	VARCHAR(10)	NULL	Emoji opcional para identificar visualmente la categoría
es_predeterminada	BOOLEAN	DEFAULT false	true para las 8 categorías del sistema; false para las del usuario
created_at	TIMESTAMPTZ	DEFAULT NOW()	Fecha de creación del registro

Tabla: transacciones

Columna	Tipo	Restricciones	Descripción
id	UUID	PK, DEFAULT gen_random_uuid()	Identificador único de la transacción
usuario_id	UUID	NOT NULL, FK auth.users	Usuario propietario de la transacción
categoria_id	UUID	NOT NULL, FK categorias(id)	Categoría a la que pertenece la transacción
monto	DECIMAL(12,2)	NOT NULL, CHECK > 0	Monto de la transacción en pesos colombianos
tipo	VARCHAR(10)	NOT NULL, CHECK IN ('ingreso','gasto')	Tipo de movimiento financiero
descripcion	VARCHAR(200)	NULL	Descripción opcional del movimiento
fecha	DATE	NOT NULL	Fecha en que ocurrió la transacción
created_at	TIMESTAMPTZ	DEFAULT NOW()	Fecha y hora de registro en el sistema

Tabla: presupuestos

Columna	Tipo	Restricciones	Descripción
id	UUID	PK, DEFAULT gen_random_uuid()	Identificador único del presupuesto
usuario_id	UUID	NOT NULL, FK auth.users	Usuario propietario del presupuesto
categoria_id	UUID	NOT NULL, FK categorias(id)	Categoría a la que aplica el límite
monto_limite	DECIMAL(12,2)	NOT NULL, CHECK > 0	Monto máximo de gasto permitido para el mes
mes	SMALLINT	NOT NULL, CHECK BETWEEN 1 AND 12	Mes al que aplica el presupuesto (1=Enero … 12=Diciembre)
anio	SMALLINT	NOT NULL	Año al que aplica el presupuesto
created_at	TIMESTAMPTZ	DEFAULT NOW()	Fecha y hora de creación del presupuesto
UNIQUE	—	(usuario_id, categoria_id, mes, anio)	Un solo presupuesto por categoría por mes por usuario

5.2. Categorías predeterminadas del sistema

Al registrarse un nuevo usuario, se insertan automáticamente las siguientes 8 categorías con usuario_id = NULL y es_predeterminada = true:

N°	Nombre	Tipo	Ícono
1	Comida	gasto	□
2	Transporte	gasto	□
3	Educación	gasto	□
4	Salud	gasto	□
5	Entretenimiento	gasto	□
6	Ropa	gasto	□
7	Servicios	gasto	□
8	Otros	gasto	□

6. Requisitos Funcionales

RF-01	Autenticación	El sistema debe permitir a un nuevo estudiante crear una cuenta con nombre completo, correo electrónico y contraseña.	Prioridad: Alta
1	El formulario valida que el correo tenga formato válido antes de enviar		
2	La contraseña tiene mínimo 8 caracteres — validado en el cliente y en Supabase Auth		
3	Si el correo ya está registrado, se muestra mensaje de error descriptivo en español		
4	Al registrarse exitosamente, el usuario inicia sesión automáticamente y es redirigido al dashboard		
5	La contraseña se almacena como hash bcrypt por Supabase Auth — nunca en texto plano		

RF-02	Autenticación	El sistema debe permitir iniciar sesión con correo y contraseña y gestionar la sesión activa mediante tokens JWT.	Prioridad: Alta
1	Las credenciales incorrectas muestran un mensaje de error genérico sin indicar cuál campo falló		
2	El token JWT se almacena y gestiona automáticamente por el SDK de Supabase		
3	La sesión persiste entre recargas de página hasta que el usuario cierre sesión o el token expire		
4	ProtectedRoute.jsx redirige al login si se intenta acceder a una ruta protegida sin sesión		
5	El logout destruye la sesión de Supabase y redirige al login		

RF-03	Transacciones	El sistema debe permitir registrar, editar y eliminar transacciones financieras (ingresos y gastos) con monto, tipo, categoría, fecha y descripción.	Prioridad: Alta
1	El formulario incluye: monto (número positivo > 0), tipo (Ingreso/Gasto), categoría, fecha y descripción opcional		
2	Al cambiar el tipo en el formulario, el select de categorías se actualiza mostrando solo las del tipo seleccionado		
3	La transacción se guarda en Supabase con el usuario_id del usuario autenticado		
4	Al editar, el formulario se pre-rellena con los datos actuales de la transacción		
5	La eliminación requiere confirmación en un modal antes de proceder		
6	Solo el propietario puede editar o eliminar sus transacciones (RLS)		

RF-04	Transacciones	El sistema debe mostrar el historial de transacciones filtrado por mes y año con totales calculados.	Prioridad: Alta
1	Selector de mes/año que filtra el historial mostrado — por defecto el mes actual		
2	Listado ordenado por fecha descendente con columnas: Fecha, Descripción, Categoría, Tipo, Monto		
3	Ingresos destacados en verde y gastos en rojo para diferenciación visual inmediata		
4	Totales visibles al pie: Total Ingresos, Total Gastos y Balance Neto del período		
5	Estado vacío con mensaje amigable cuando no hay transacciones en el mes seleccionado		

RF-05	Transacciones	El sistema debe permitir filtrar transacciones por texto libre, tipo y categoría, y paginar el historial.	Prioridad: Media
1	Campo de búsqueda que filtra por descripción en tiempo real sobre los datos ya cargados		
2	Filtros combinables: texto + tipo + categoría aplicados simultáneamente		
3	Paginación con opciones de 10, 25 y 50 registros por página		
4	Botón 'Limpiar filtros' que restaura el listado completo del mes		

RF-06	Transacciones	El sistema debe permitir exportar el historial de transacciones del mes actual a un archivo CSV.	Prioridad: Media
1	El botón 'Exportar CSV' genera y descarga el archivo desde el cliente (sin petición al servidor)		
2	El CSV incluye encabezados: Fecha, Descripción, Categoría, Tipo, Monto		
3	El archivo exportado refleja los filtros activos al momento de la exportación		
4	El nombre del archivo incluye el mes y año (ej: transacciones-marzo-2026.csv)		

RF-07	Categorías	El sistema debe incluir 8 categorías predeterminadas y permitir crear, editar y eliminar categorías personalizadas.	Prioridad: Alta
1	Las categorías predeterminadas se muestran en sección separada sin botones de editar/eliminar		
2	El usuario puede crear categorías con nombre único, tipo e ícono emoji opcional		
3	Solo se puede eliminar una categoría personalizada que no tenga transacciones asociadas		
4	Si tiene transacciones, se muestra mensaje explicativo y no se procede con la eliminación		
5	Las nuevas categorías aparecen inmediatamente en el select de transacciones		

RF-08	Presupuesto	El sistema debe permitir definir presupuestos mensuales por categoría y calcular automáticamente el porcentaje consumido.	Prioridad: Alta
1	Solo se puede crear un presupuesto por combinación usuario + categoría + mes + año		
2	El select de categorías en presupuestos muestra solo las de tipo 'gasto'		
3	El porcentaje consumido se calcula en tiempo real consultando las transacciones del mes		
4	Se puede editar el monto límite de un presupuesto existente		
5	Se puede eliminar un presupuesto con confirmación previa		

RF-09	Presupuesto	El sistema debe mostrar alertas visuales automáticas cuando el gasto supere el 80% y el 100% del límite del presupuesto.	Prioridad: Alta
1	Barra de progreso por presupuesto con tres estados: azul 0–79%, amarillo 80–99%, rojo 100%+		
2	Al superar el 80%: badge amarillo con texto '¡Cerca del límite!'		
3	Al superar el 100%: badge rojo con texto 'Límite superado'		
4	Los porcentajes se recalculan automáticamente al registrar una nueva transacción		
5	Los presupuestos críticos (rojo) aparecen primero en la lista		

RF-10	Dashboard	El sistema debe mostrar 4 tarjetas de resumen financiero del mes con totales en tiempo real.	Prioridad: Alta
1	Tarjetas: Total Ingresos (verde), Total Gastos (rojo), Balance Neto y N° de transacciones		
2	El balance muestra color verde si es positivo y rojo si es negativo		
3	Las cifras se muestran con formato de moneda: $ 1.250.000		
4	Selector de mes/año que actualiza todas las tarjetas y gráficas simultáneamente		

RF-11	Dashboard	El sistema debe renderizar tres gráficas interactivas con Recharts usando datos reales de la base de datos.	Prioridad: Alta
1	Gráfica de torta: distribución de gastos del mes por categoría con tooltips de nombre, monto y porcentaje		

2	Gráfica de barras: ingresos y gastos de los últimos 6 meses con línea de balance neto superpuesta
3	Gráfica de línea: evolución del balance neto con área coloreada según signo (positivo/negativo)
4	Si no hay datos en el período, se muestra estado vacío con mensaje informativo
5	Al cambiar el selector de mes, las gráficas se destruyen y reinicializan con los nuevos datos

RF-12	Dashboard	El sistema debe mostrar el top 5 categorías con más gasto del mes y el estado global de presupuestos.	Prioridad: Media
1	Lista con las 5 categorías con mayor gasto: nombre, monto y barra de progreso proporcional		
2	Contadores de estado de presupuestos: cuántos en verde, amarillo y rojo		
3	Enlace directo a la sección de presupuestos desde el dashboard		

RF-13	Perfil	El sistema debe permitir al usuario autenticado actualizar su nombre, correo y contraseña.	Prioridad: Media
1	Sección 'Mis datos': campos editables de nombre y correo con validación de formato		
2	Sección 'Cambiar contraseña': campos contraseña actual, nueva y confirmar nueva		
3	La contraseña actual se verifica antes de permitir el cambio		
4	Ambas secciones muestran mensajes de éxito o error de forma independiente		
5	Botón de mostrar/ocultar contraseña en los campos de texto oculto		

7. Requisitos No Funcionales

RNF-01	Rendimiento
1	El tiempo de carga inicial de la aplicación React no debe superar 3 segundos en una conexión normal.
2	Las consultas a Supabase deben resolverse en menos de 1 segundo para listas de hasta 500 registros.
3	La exportación CSV debe completarse en menos de 2 segundos para hasta 1.000 transacciones.
4	Las gráficas del dashboard deben renderizarse en menos de 1.5 segundos tras recibir los datos.

RNF-02	Seguridad
1	Todas las contraseñas se almacenan como hash bcrypt gestionado por Supabase Auth — nunca en texto plano.
2	Todas las tablas tienen Row Level Security (RLS) habilitado con políticas que restringen el acceso por usuario autenticado.
3	El archivo .env.local con las credenciales de Supabase nunca debe subirse al repositorio Git.
4	La anon key de Supabase solo permite operaciones dentro de los límites definidos por las políticas RLS.
5	Los tokens JWT expiran automáticamente — Supabase gestiona la renovación de sesiones.
6	ProtectedRoute.jsx verifica la sesión activa antes de renderizar cualquier página privada.

RNF-03	Usabilidad
1	Un usuario nuevo debe poder registrar su primera transacción en menos de 3 minutos desde el primer login.
2	Todos los mensajes de error y éxito deben estar en español claro, sin términos técnicos visibles al usuario.
3	Los formularios deben tener focus automático en el primer campo al abrirse.
4	Los botones e inputs deben tener tamaño mínimo de 44px en dispositivos móviles.
5	Se debe mostrar un indicador de carga (spinner) en todas las operaciones que tarden más de 500ms.

RNF-04	Compatibilidad y Responsive
1	La app debe funcionar correctamente en las últimas versiones de Chrome, Firefox y Safari.
2	El diseño debe ser completamente funcional en resoluciones de 375px (iPhone SE), 768px (tablet) y 1280px (escritorio).
3	La tabla de transacciones debe permitir scroll horizontal en pantallas pequeñas sin romper el layout.
4	El menú de navegación debe colapsar en un menú hamburguesa en pantallas menores a 768px.

RNF-05	Mantenibilidad
1	El código debe seguir la arquitectura de carpetas definida en la sección 4.2 (pages, hooks, services, utils).
2	Cada componente React debe tener una única responsabilidad y no superar las 200 líneas de código.
3	Los custom hooks deben encapsular toda la lógica de estado y efectos, sin mezclar con la UI.

4	Los services solo ejecutan queries a Supabase y retornan datos — sin lógica de UI ni estado.
5	El repositorio debe seguir la convención de commits definida en el contrato: feat:, fix:, docs:, etc.

RNF-06	Disponibilidad
1	La plataforma debe estar disponible y funcional durante las sesiones de Sprint Review y pruebas con usuarios.
2	El entorno de desarrollo debe poder levantarse desde cero siguiendo únicamente el README en menos de 10 minutos.
3	El script de base de datos (migrations/001_initial_schema.sql + policies.sql) debe ejecutarse sin errores en un proyecto Supabase limpio.

8. Supabase — Servicios y Funciones RPC

8.1. Tablas expuestas vía API REST

Tabla	Operaciones permitidas	Política RLS activa
categorias	SELECT	Lectura: categorías del sistema (usuario_id IS NULL) + propias del usuario autenticado
categorias	INSERT, UPDATE, DELETE	Solo sobre filas donde usuario_id = auth.uid() Y es_predeterminada = false
transacciones	SELECT, INSERT, UPDATE, DELETE	Solo sobre filas donde usuario_id = auth.uid()
presupuestos	SELECT, INSERT, UPDATE, DELETE	Solo sobre filas donde usuario_id = auth.uid()

8.2. Funciones RPC del Dashboard

Función	Parámetros	Retorna	Descripción
get_resumen_mes	p_usuario UUID, p_mes INT, p_anio INT	{ total_ingresos, total_gastos, balance, num_transacciones }	Totales del mes para las tarjetas del dashboard
get_gastos_por_categoria	p_usuario UUID, p_mes INT, p_anio INT	Array: { categoria, monto_total, porcentaje }	Datos para la gráfica de torta
get_evolucion_mensual	p_usuario UUID	Array 6 meses: { mes, anio, ingresos, gastos, balance }	Datos para la gráfica de barras y línea

get_top_categorias	p_usuario UUID, p_mes INT, p_anio INT, p_limite INT	Array: { categoria, monto, porcentaje_del_total }	Top N categorías con más gasto
get_presupuestos_con_gasto	p_usuario UUID, p_mes INT, p_anio INT	Array: { ...presupuesto, monto_gastado, porcentaje_consumido }	Presupuestos con gasto real calculado

8.3. Supabase Auth — Flujo de autenticación

Acción	Método SDK	Respuesta esperada
Registrar usuario	supabase.auth.signUp({ email, password })	{ data: { user, session }, error: null }
Iniciar sesión	supabase.auth.signInWithPassword({ email, password })	{ data: { user, session }, error: null }
Cerrar sesión	supabase.auth.signOut()	{ error: null }
Obtener sesión activa	supabase.auth.getSession()	{ data: { session }, error: null }
Escuchar cambios	supabase.auth.onAuthStateChange(callback)	Evento: SIGNED_IN, SIGNED_OUT, TOKEN_REFRESHED
Actualizar correo	supabase.auth.updateUser({ email: nuevoCorreo })	{ data: { user }, error: null }
Cambiar contraseña	supabase.auth.updateUser({ password: nuevaPass })	{ data: { user }, error: null }

9. Criterios de Aceptación Global del Proyecto

El proyecto se considerará completado y apto para la entrega final cuando cumpla la totalidad de los siguientes criterios:

#	Criterio	Cómo se verifica
1	Las 26 historias de usuario del backlog tienen estado 'Terminado' en GitHub Projects	Revisión del tablero Kanban del repositorio
2	Todos los flujos principales funcionan sin errores en Chrome y Firefox	Prueba manual siguiendo la guía de testers
3	Las 3 gráficas de Recharts renderizan datos reales y responden al selector de mes	Demo en Sprint Review del Sprint 4
4	Las alertas de presupuesto al 80% y 100% se activan correctamente	Prueba con datos controlados de gasto

5	La exportación CSV genera un archivo válido con todos los campos correctos	Abrir el CSV en Excel o Google Sheets
6	Las políticas RLS impiden que un usuario acceda a datos de otro usuario	Prueba con 2 cuentas distintas intentando acceso cruzado
7	La app funciona en resoluciones de 375px y 1280px sin errores de layout	Inspección en DevTools con Device Mode
8	El README permite instalar y correr el proyecto desde cero sin pasos adicionales	Instalación en un equipo limpio sin intervención del equipo
9	Mínimo 8 estudiantes universitarios completaron el flujo de prueba con satisfacción ≥ 3.5/5	Resultados de la encuesta Google Forms del Sprint 5
10	El diagrama ER, la documentación de servicios y el manual de usuario están entregados en /docs	Revisión del repositorio en la entrega final

10. Restricciones y Decisiones Técnicas

Restricción	Decisión	Justificación
Sin backend propio	Todo el backend se delega a Supabase	Reduce complejidad operativa y tiempo de desarrollo en el contexto académico
Sin App móvil nativa	La app es web-first con diseño responsive	Accesible desde cualquier dispositivo sin instalación; alcance del MVP
Sin vinculación bancaria	El registro de transacciones es 100% manual por el usuario	Evita complejidad legal y técnica de integración con APIs bancarias
Sin notificaciones push	Las alertas son exclusivamente visuales dentro de la app	Fuera del alcance del MVP; puede implementarse en versiones futuras
Sin modo offline	La app requiere conexión a internet para funcionar	Supabase depende de conexión; sin service workers en el MVP
Sin roles de admin	Un solo tipo de usuario en el MVP	El alcance académico no requiere panel administrativo

Nota final:

Las presentes especificaciones técnicas constituyen el Anexo 2 del Contrato de Desarrollo de Software N° FINU-2026-001 suscrito entre FinanzasU S.A.S. (EL CLIENTE) y Rise (EL DESARROLLADOR). Cualquier cambio, adición o eliminación de requisitos aquí definidos deberá acordarse por escrito entre el Representante Legal del Cliente (Juan Camilo Triana) y el Representante Legal del Desarrollador (Deibyd Castillo) antes de ser implementado, y podrá implicar ajustes en el cronograma y el valor del contrato.